

Bank Transaction Fraud Detection System

Assessment type: Individual programming assignment

Weighting: 50%

Due date and time: Week 9, 18 Sep 2026, 11:59 PM

Submission format: Submit two files via Canvas:

1. Report (Word document) Filename: <StudentName> - Task 2.docx
2. Python Program (ZIP archive)
Filename: <StudentName> - Task 2.zip
The ZIP file should include all required Python source files, test program, data files, configuration files (if applicable), and any supporting resources required to run the program. Use relative file paths so the program can be executed after extraction without modification.

1. Background

Banks process large numbers of financial transactions every day. Although most transactions are legitimate, some may indicate potentially fraudulent activity. Fraud analysts examine transaction records to identify unusual patterns and flag suspicious transactions or accounts for further investigation.

In this assessment, you will develop a Python program that analyses fictional bank transaction records. Your program will read transaction data from a file, validate the records, store valid transactions, apply fraud-detection rules, and display suspicious activity.

The assessment focuses on fundamental Python programming skills, including:

- File handling
- Data validation
- Lists and dictionaries
- Loops and conditional statements
- Functions
- Data analysis
- Testing
- Program documentation

Important: This is a fictional scenario created for assessment purposes. All banks, accounts, customers, transactions, and financial information used in this assessment must be fictional.

2. Assessment Objectives

This assessment evaluates your ability to:

- Read and process structured data files.
- Validate input data.

- Store and organise data using appropriate Python data structures.
- Use loops and conditional statements to analyse data.
- Develop modular Python programs.
- Apply predefined rules to identify suspicious activity.
- Test program functions using valid and invalid inputs.
- Explain and justify your program design.

3. Transaction Data

Your program must process transaction records in the following format:

timestamp | transaction_id | account_id | transaction_type | amount | location | status

3.1 Example Records

2026-07-11 09:15:23 | T1001 | A10025 | TRANSFER | 850.00 | Brisbane | APPROVED

2026-07-11 09:18:40 | T1002 | A10025 | TRANSFER | 6500.00 | Brisbane | APPROVED

2026-07-11 09:20:12 | T1003 | A10025 | ONLINE_PURCHASE | 120.00 | Sydney | DECLINED

2026-07-11 09:22:05 | T1004 | A20318 | CASH_WITHDRAWAL | 500.00 | Gold Coast | APPROVED

3.2 Field Formats

The following formats should be used:

- **timestamp:** YYYY-MM-DD HH:MM:SS
Example: 2026-07-11 09:15:23
- **transaction_id:** The letter T followed by four digits
Example: T1001
- **account_id:** The letter A followed by five digits
Example: A10025
- **transaction_type:** One of the approved transaction types listed below
- **amount:** A positive numeric value with up to two decimal places
Example: 850.00
- **location:** A non-empty location name
Example: Brisbane
- **status:** One of the approved status values listed below

Students implementing the **Enhanced Data Validation** advanced feature should validate these formats and identify records that do not comply.

3.3 Valid Transaction Types

The **transaction_type** field may contain:

- TRANSFER
- CARD_PAYMENT
- CASH_WITHDRAWAL

- ONLINE_PURCHASE
- DIRECT_DEBIT

3.4 Valid Status Values

The status field may contain:

- APPROVED
- DECLINED
- PENDING

4. Functional Requirements (Core Features – Pass/Credit)

Students must complete all core features.

4.1 Read the Transaction File

Your program must:

- Read transaction records from a text or CSV file.
- Process records in the order they appear.
- Handle a missing, empty, or unreadable file without terminating unexpectedly.
- Display an appropriate error message if the file cannot be processed.

JSON input is **not** required for the core features.

4.2 Perform Basic Data Validation

Validate each transaction before including it in the analysis. For the core requirements, your program must check that:

- The record contains the correct number of fields.
- The transaction ID is not empty.
- The account ID is not empty.
- The transaction type is valid.
- The status value is valid.
- The amount is numeric.
- The amount is greater than zero.

Invalid records must not be included in the fraud analysis.

For each invalid record, display a simple message identifying:

- The line number (or record number).
- The reason the record was rejected.

Detailed timestamp validation, duplicate transaction detection, and exporting invalid records are not required for the core features.

4.3 Store Valid Transactions

After validation, store all valid transaction records in memory using appropriate Python data structures.

Suitable approaches include:

- A list of dictionaries
- A dictionary grouped by account ID
- Another appropriate combination of lists and dictionaries

The selected data structure should allow the program to:

- Store valid transaction records
- Access individual transaction details
- Group or search transactions by account ID
- Apply the required fraud-detection rules efficiently

4.4 Detect Suspicious Activity

Your program must implement the following three fraud-detection rules.

Rule 1 – High-Value Transactions

Flag any transaction that:

- Has a status of APPROVED, and
- Has an amount greater than \$5,000.

For each flagged transaction, display:

- Transaction ID
- Account ID
- Transaction amount
- Detection rule
- Reason the transaction was flagged

Example: Transaction T1002 was flagged because the approved amount of \$6,500 exceeded the \$5,000 threshold.

Rule 2 – Repeated Declined Transactions

Flag an account when it has **four or more declined transactions**.

For each flagged account, display:

- Account ID
- Number of declined transactions
- Relevant transaction IDs

- Detection rule
- Reason the account was flagged

For the core requirements, declined transactions may be counted across the complete transaction file.

Rule 3 – Multiple Locations

Flag an account when transactions occur in three or more different locations.

For each flagged account, display:

- Account ID
- Number of different locations
- Locations involved
- Relevant transaction IDs
- Detection rule
- Reason the account was flagged

For the core requirements, locations may be counted across the complete transaction file.

4.5 Display Suspicious Activity

Display each suspicious transaction or account clearly.

The output must include:

- Account ID
- Transaction ID (where applicable)
- Detection rule triggered
- Relevant transaction details
- A brief explanation of why the activity was flagged

The output may be displayed in the terminal or console. Summary statistics are **not** required for the core features.

5. Advanced Features (Distinction / High Distinction)

Advanced features extend the completed core program and must not replace the core requirements.

Students may choose any of the advanced features below. The features are independent and do not need to be completed in order.

- To be considered for a Distinction, students must successfully implement at least TWO advanced features.
- To be considered for a High Distinction, students must successfully implement at least FOUR advanced features.

Each advanced feature should demonstrate a meaningful extension to the core program. Simple cosmetic changes or minor modifications will not normally satisfy the requirements for an advanced feature. The quality, correctness, integration, testing, and documentation of the selected advanced features will also be considered.

5.1 Advanced Feature 1 – Enhanced Data Validation

- First, validate each field against the requirements specified in **Section 3.2 Field Formats**, including:
 - Timestamp
 - Transaction ID
 - Account ID
 - Transaction type
 - Amount
 - Location
 - StatusIdentify and reject any records that do not comply with these formats.
- Then extend the validation process by implementing at least **two additional validation checks**, such as:
 - Detecting duplicate transaction IDs
 - Identifying missing or additional fields
 - Detecting duplicate transaction records
 - Checking for leading or trailing whitespace
 - Identifying inconsistent capitalisation
 - Storing invalid records separately
 - Exporting invalid records to a text or CSV file

The program must continue processing valid records when invalid records are found.

For each rejected record, display or store:

- The relevant line or record number
- The invalid field or value
- A clear reason for rejection

5.2 Advanced Feature 2 – Time-Based Fraud Detection

Implement at least one additional fraud-detection rule.

Examples include:

- Four or more declined transactions within 30 minutes
- Five or more transactions from one account within one hour
- Three or more transactions just below the \$5,000 threshold
- Multiple large cash withdrawals within one day
- A high-value transaction shortly after repeated declined transactions
- Transactions from different locations within a short period

The additional rule must:

- Use at least two transaction fields
- Be implemented as a separate function
- Explain why the activity is suspicious
- Be tested using both suspicious and normal data
- Be documented in the report

5.3 Advanced Feature 3 – Advanced Transaction Storage and Management

Extend the storage or management of transaction data by implementing one or more of the following:

- Organise transactions using multiple or more advanced linked data structures
- Store suspicious transactions separately from valid transactions
- Implement functions to search, filter, sort, or update stored records
- Save processed transactions or suspicious activity to a CSV or JSON file

The selected feature should be fully integrated into the program, tested, and explained in the report.

5.4 Advanced Feature 4 – Configurable Fraud Rules

Store fraud-detection settings in a JSON or CSV configuration file. The configuration file may contain:

- High-value transaction threshold
- Declined transaction threshold
- Multiple-location threshold
- Time-window threshold
- Rule name
- Severity level

The program should automatically load the configuration settings and apply them during fraud detection.

5.5 Advanced Feature 5 – Customer Account Analysis

Read a second file containing fictional customer account information.

Example:

account_id | account_type | home_location | daily_limit | account_status

Use the account information to identify one or more of the following:

- Transactions exceeding the daily limit
- Transactions outside the home location
- Transactions from dormant or suspended accounts
- Unusual activity based on the account type

5.6 Advanced Feature 6 – Database and Investigation Management

Store suspicious transactions or investigation records in an SQL or NoSQL database.

Possible features include:

- Add suspicious records
- View stored records
- Search by account ID
- Search by transaction ID
- Search by detection rule
- Filter by transaction type, location, or status
- Update investigation status
- Prevent duplicate records

You may also implement an interactive investigation management menu.

5.7 Advanced Feature 7 – Summary Statistics and Reporting

Generate a summary report that includes:

- Total number of records processed
- Total number of valid records
- Total number of invalid records
- Total number of approved transactions
- Total number of declined transactions
- Total value of approved transactions
- Total number of suspicious transactions
- Total number of suspicious accounts
- Number of high-value transaction alerts
- Number of repeated-decline alerts
- Number of multiple-location alerts

An account that triggers multiple fraud-detection rules should only be counted once as a suspicious account.

In addition, implement **one** of the following:

- Export the summary to a CSV file
- Export the summary to an Excel file
- Export the summary to a PDF report

Students may also propose an alternative advanced feature that demonstrates a comparable level of complexity and extends the core functionality. Any alternative feature should be discussed with the course coordinator before implementation.

6. Testing Requirements

Submit a separate Python testing file named: test.py

The testing file must run without requiring manual input and should verify that the program functions correctly under both normal and invalid conditions.

6.1 Test Data

Create your own transaction dataset or extend the supplied sample data. The dataset should contain sufficient records to demonstrate that all core features operate correctly.

Your test data must include:

- Normal approved transactions
- At least one approved transaction greater than \$5,000
- At least one account with four or more declined transactions
- At least one account with transactions in three or more locations
- At least one invalid transaction record
- At least one normal account that should not be flagged

6.2 Core Testing

Your tests should verify the behaviour of individual functions, where appropriate, as well as the overall program.

The tests must include:

- Reading a valid transaction file
- Handling a missing file
- Handling an empty file
- Rejecting a record with an incorrect number of fields
- Rejecting an invalid transaction type
- Rejecting an invalid status value
- Rejecting a non-numeric amount
- Rejecting a zero or negative amount
- Detecting a high-value approved transaction
- Detecting repeated declined transactions
- Detecting transactions from multiple locations
- Confirming that a normal account is not incorrectly flagged

Students should also test any advanced features implemented.

The tests should include a combination of:

- Valid and invalid inputs
- Normal and suspicious transactions
- Boundary conditions, where appropriate

Each test should identify:

- The function or feature being tested
- Test input
- Expected result
- Actual result
- Pass or Fail outcome

The test cases described in the report should correspond to those implemented in test.py.

7. Programming and Code Quality Requirements

Your program should:

- Use meaningful variable, function, and file names.
- Follow consistent Python coding conventions and formatting.
- Organise the solution into reusable and well-structured functions.
- Use parameters and return values appropriately.
- Include appropriate comments and/or docstrings.
- Validate input and handle errors gracefully without crashing.
- Avoid unnecessary duplication of code.
- Produce clear, consistent, and user-friendly output.
- Use relative file paths where appropriate.
- Be easy to read, test, maintain, and extend.

Your solution should include functions for tasks such as:

- Reading the transaction file
- Validating transaction records
- Storing valid transactions
- Detecting suspicious activity
- Displaying results

You may implement separate functions for each fraud-detection rule or another well-structured modular design.

8. Recommended Development Workflow

The following workflow is recommended to help you develop your solution incrementally:

1. Review the assessment requirements and prepare the transaction data.
2. Read the transaction records from the input file.
3. Validate each transaction and identify invalid records.
4. Store valid transactions using appropriate Python data structures.
5. Implement and test each fraud-detection rule individually.
6. Display suspicious transactions and accounts with appropriate explanations.
7. Implement and test any selected advanced features.
8. Develop and execute test.py to verify the required functionality.
9. Complete the report, including testing results and the reflection on AI use.
10. Review, refine, and document your program before submission.

Following an incremental development approach will make your program easier to test, debug, and maintain.

9. Submission format

Please make sure to submit **two files** through Canvas. In keeping with the University of the Sunshine Coast policy, all assignments are to be submitted in Canvas via Turnitin.

9.1 Report in Word document

Prepare a professional report and submit it as a **Microsoft Word document (.docx, not PDF)**. The file must be named: **<student name> - Task 2.docx**

The report is to be submitted as a Microsoft Word document (.docx) using A4 page size, Calibri 11 pt (recommended), Arial 11 pt, or Times New Roman 12 pt, with single (1.0) line spacing and 2.54 cm (1 inch) margins on all sides. The suggested report length is approximately 8–12 pages, excluding the Title Page and Table of Contents.

When including short code snippets, use a monospaced font such as Consolas 10 pt (recommended), Courier New 10 pt, or Lucida Console 10 pt. Include only short code examples where necessary to support your discussion, as the complete Python program is submitted separately. The suggested length is provided as a guide only; the quality, clarity, organisation, and completeness of the report are more important than its exact length.

Your report should include the following sections:

- **Title Page**
 - Course code and course name
 - Assessment title
 - Student name
 - Student ID
 - Tutor name
- **Table of Contents**
- **1. Introduction:** Briefly introduce the purpose of the program and the problem being solved.
- **2. Problem Description:**
 - The purpose of your program
 - The main functionality provided
 - The transaction data used
 - Any assumptions made
 - Any known limitations
- **3. Program Design:** Explain the overall design of your solution. Include:
 - The overall program structure
 - Major functions and their purpose
 - Data structures used (for example, lists or dictionaries)
 - Pseudocode or flowcharts illustrating your solutionYou may include your initial design, final design, or both.
- **4. Libraries and Dependencies:** Identify any Python libraries used in your program. For each library, include:
 - Library name
 - Purpose

- Installation instructions (if required). If no additional libraries are required beyond the Python Standard Library, state this clearly.
- **5. Fraud Detection Rules:** Explain how the three core fraud-detection rules were implemented. If advanced features were completed, briefly explain how they extend the core functionality.
- **6. Advanced Features**
If you attempted any advanced features, concisely explain:
 - Which advanced feature(s) you selected
 - How each feature was implemented; any limitations or incomplete elements
 - How each feature extends the core program
 - Any additional data files, libraries, configuration files, or database components used

Include screenshots, diagrams, short code examples, or sample outputs where appropriate. If no advanced features were attempted, state this clearly.

- **7. Testing:** Include:
 - A test plan
 - Detailed test cases
 - Test inputs
 - Expected outputs
 - Actual outputs
 - Test results (Pass/Fail)

Screenshots of selected program output may also be included where appropriate.

- **8. Challenges and Future Improvements**

Briefly discuss:

- Challenges encountered during development
- How the challenges were addressed
- Possible future improvements to the program

- **9. Reflection on Artificial Intelligence (AI) Use**

Briefly explain how AI tools were used during the planning, development, debugging, testing, documentation, or report-writing stages.

Include:

- The AI tool(s) used
- At least three detailed prompts and a brief summary of the corresponding outputs
- How each output was applied, modified, or rejected
- Any errors, limitations, or unsuitable suggestions identified
- How AI-generated content was checked, tested, and verified

Where relevant, screenshots of AI prompts and outputs can be included to illustrate your reflection.

9.2 Python Program

Compress your Python program into a single ZIP file.

The file must be named: **<student name> - Task 2.zip**

The ZIP file should include all files required to run your program, for example:

- main.py
- test.py
- Sample transaction data file(s)
- Configuration file(s) (if used)
- Customer account data file(s) (if used)

- Database file(s) (if used)
- Exported reports or charts (if applicable)
- Any additional supporting files

Your program should:

- Execute without modification after extraction.
- Open files from the same folder as the Python program using relative file paths.
- **Include appropriate comments and documentation throughout the code.**

10. Artificial Intelligence (AI) Use

Generative Artificial Intelligence (AI) tools (e.g., ChatGPT, Microsoft Copilot, GitHub Copilot, Claude, Gemini, or similar tools) **may be used** to support learning and software development for this assessment. AI should be used to **assist** your learning and problem solving, **not replace** your own analysis, programming, testing, or understanding.

Examples of appropriate AI use include:

- Brainstorming solution ideas
- Explaining Python concepts, syntax, or error messages
- Debugging code or suggesting improvements
- Improving code readability or documentation
- Assisting with report writing and editing
- Generating fictional sample or test data
- Explaining algorithms, libraries, or programming techniques

Students are responsible for ensuring that:

- All submitted work complies with the University's Academic Integrity Policy.
- They understand, review, test, debug, and verify all AI-generated content before submission.
- They can explain and justify all submitted code, algorithms, design decisions, and report content if requested.
- All submitted code satisfies the assessment requirements, regardless of whether AI was used.
- AI-generated content is accurate, appropriate, and free from fabricated, incorrect, or misleading information.
- Only fictional assessment data is uploaded to public AI tools.

Students **must not**:

- Submit AI-generated work without reviewing and understanding it.
- Use AI as a substitute for their own analysis, programming, testing, or learning.
- Present AI-generated content as their own original work without appropriately disclosing its use in the report.
- Use AI to fabricate testing results, program outputs, experimental findings, or report content.
- Submit code or report content that they cannot explain or justify.

Students must complete the **Reflection on Artificial Intelligence (AI) Use** section in their report, describing how AI was used during the planning, development, debugging, testing, documentation, and report-writing stages.

The use of AI does not replace the University's Academic Integrity Policy. Students remain responsible for ensuring that all submitted work is accurate, original where required, and complies with all assessment requirements.